

Invoice Generator

Future Scope

Prioritised backlog from a review of the codebase - 26 August 2026

Twelve items, ordered by what would move the product furthest rather than what is quickest. Each entry says what exists today, why it matters, and where the work would start - so a point can be picked up cold months from now. Nothing here is urgent enough to stop what you are doing; items 1 to 3 are the ones that compound.

PRIORITY 1

Do these first

Each one unblocks the next. PDF generation gates email, and email gates the whole get-paid loop.

1. Real PDF generation

TODAY	The "Download PDF" button calls <code>window.print()</code> - the browser's print dialog, not a PDF. See <code>src/app/share/[publicSlug]/download-button.tsx</code> .
WHY IT MATTERS	Output depends on the client's own browser and print settings (margins, "1/2 localhost:3000" headers). Rough on mobile. Nothing to attach to an email. No immutable archive of what was actually sent.
HEAD START	<code>@vercel/blob</code> is already a dependency, and <code>BLOB_READ_WRITE_TOKEN</code> is already stubbed out in <code>.env.example</code> .
APPROACH	Render server-side at finalize time, store the blob, serve that file. Finalize already freezes a content snapshot, so it is the natural hook to hang this on.

2. Email delivery

TODAY	No email code anywhere in the repo. Sharing is link-only, so you copy a URL into WhatsApp by hand.
WHY IT MATTERS	The get-paid loop is entirely manual. Every ingredient already exists: <code>dueDate</code> , the Payment ledger, a derived overdue state, and <code>DocumentActivity</code> logging views.
APPROACH	A "send invoice" action plus automated reminders on overdue. Depends on item 1 - you want a real PDF to attach.

3. GST that matches Indian invoicing

TODAY	<code>TaxMode</code> is <code>NONE</code> or <code>PERCENTAGE</code> - one rate, one label. A single "GST 18%" line on the document.
WHY IT MATTERS	Indian GST needs a <code>CGST+SGST</code> split intra-state and <code>IGST</code> inter-state, decided by supplier state versus place of supply, plus <code>HSN/SAC</code> codes per line item. A single combined line will not reconcile against <code>GSTR-1</code> or satisfy a client's accountant.
HEAD START	<code>GSTIN</code> fields already exist on <code>BusinessProfile.taxId</code> and <code>Client.taxId</code> , and <code>Client.country</code> is there.
CAVEAT	Confirm the exact requirement with your CA before modelling it. This is a schema change, so it is far cheaper now than after a few hundred invoices exist.

PRIORITY 2

Correctness gaps

Not visible day to day, but each one is a silent failure waiting to happen.

4. No error boundaries anywhere

TODAY	Twelve loading.tsx files exist, but there is no error.tsx, not-found.tsx or global-error.tsx anywhere in src/app.
WHY IT MATTERS	Server actions throw useful messages and the editor catches those into local state - but a render-time throw takes out the whole route with a generic error page. src/lib/types.ts explicitly warns that a hand-edited row could violate the content JSON shape, which is exactly that case.

5. Zero automated tests

TODAY	No test runner in package.json and no test files in the repo.
WHY IT MATTERS	src/lib/money.ts does integer minor-unit arithmetic where a rounding regression silently corrupts invoices. Throwaway test scripts have already earned their keep - they are how the paymentSummary quirk surfaced, where a past-due partial payment is labelled "Partial" rather than "Overdue".
APPROACH	Vitest, with coverage for money.ts, document-filters.ts, serial allocation in allocate-ref.ts, and document-kinds.ts.

6. getCurrentUser() writes on every request

TODAY	It ends in a prisma.user.upsert - a database write on every cold request, for a row that changes essentially never. It is memoised per request, but that only dedupes within a single render.
APPROACH	Read first, create only if missing.

PRIORITY 3

Will bite at scale

Fine today with a handful of records. Both get worse in direct proportion to how well the business does.

7. No pagination and no search

TODAY	The documents and clients pages both findMany every row and render all of them. There is no search box on either.
WHY IT MATTERS	Fine at 2 documents, painful at 500, unusable at 5,000. Search bites sooner than pagination - finding "that Acme invoice from March" is currently Ctrl+F.
APPROACH	Both slot into the same URL-state pattern the filters already use, so the plumbing exists.

8. The dashboard is single-currency

TODAY	Totals cover the default currency only; foreign-currency invoices are excluded with a footnote underneath.
WHY IT MATTERS	If you bill international clients, the dashboard quietly under-reports what you are actually owed.
APPROACH	Either break the totals out per currency, or freeze an FX rate at finalize time and convert.

PRIORITY 4

Product depth

Worth doing once the foundations above are solid.

9. Quote lifecycle

GAP	Quotes and proformas exist and convert into invoices, but there is no accepted / rejected / expired state - so a quote past its valid-until date still looks live.
APPROACH	Client-side acceptance, mirroring the contract e-signature flow that already works.

10. Credit notes and partial refunds

GAP	The only correction available on a finalized invoice is VOID. Real accounting needs a credit note that references the original document.
-----	--

11. Recurring invoices

GAP	No retainer support. For most freelancers and agencies this is the steadiest income, and re-creating it by hand every month is pure friction.
-----	---

12. UPI payment link and QR code

GAP	The UPI ID prints as plain text that the client has to retype into their banking app.
APPROACH	A <code>upi://pay?pa=...&am=...</code> deep link plus a QR code on the document. A few lines of work, and it measurably shortens time to payment.

A structural note - this one is good news

The single-admin lock is deliberate and genuinely well-defended: enforced in `getCurrentUser`, again in the login action, and again in middleware. Worth knowing that the data model underneath is already multi-tenant, because every query is scoped by `userId`. So when this becomes a module of the larger system, the work sits in the auth layer only - it is not a data migration. That was a good call to make early.

Already logged, still open

Date hydration mismatch: the server renders dates as `DD/MM/YYYY` while an en-US browser renders `MM/DD/YYYY`, which throws a React hydration error and forces a re-render - on the public share page that clients actually see. It affects 7 call sites across 6 files and predates the recent work. Fixing it needs a decision from you on which date format client-facing documents should use.